



### Продвинутые практики в API

#### 1. Введение

На предыдущих занятиях мы прошли путь от проектирования API-контракта до его полной реализации в виде работающего микросервиса. Наш сервис функционален, но для того, чтобы он соответствовал современным промышленным стандартам, необходимо добавить несколько ключевых элементов, которые делают API не просто работающим, а надежным, удобным для изучения и простым в отладке.

Эта работа посвящена рутинной «обвязке» нашего сервиса, которая является неотъемлемой частью любого production-ready приложения.

В мире распределенных систем один пользовательский запрос может проходить через несколько микросервисов. Представьте, что произошла ошибка. Как понять, в каком из пяти сервисов она случилась и какие события ей предшествовали?

**Correlation ID** решает эту проблему, «помечая» запрос уникальным идентификатором, который передается от сервиса к сервису и пишется в логи на каждом этапе. Это позволяет восстановить всю цепочку вызовов.

**Детальное логирование** — это ваш главный инструмент при отладке и мониторинге работы приложения в реальных условиях.

Дополнительно мы добавим точку входа, которая упростит коммуникацию клиентов с нашим приложением. **HATEOAS Entry Point** делает ваше API самодокументируемым. Клиенту достаточно знать одну-единственную точку входа, чтобы дальше «путешествовать» по API, переходя по ссылкам, как человек по веб-сайту.

#### Необходимое ПО и настройка среды:

Для выполнения работы вам понадобятся полностью готовые проекты из предыдущих занятий:

##### 1. Проект-контракт (books-api-contract)

##### 2. Проект-реализация (demo-rest), в который уже подключен контракт.

Все изменения в этой работе будут производиться **только в проекте-реализации (demo-rest)**.

#### 2. Теория

##### HATEOAS и точка входа (Root Entry Point)



HATEOAS (Hypermedia as the Engine of Application State) — это принцип, согласно которому ответы от сервера должны содержать не только данные, но и ссылки (гипермедиа) на возможные следующие действия. Мы уже реализовали это для отдельных книг и авторов.

Однако у любого хорошо спроектированного HATEOAS API должна быть «главная страница» или **точка входа** — корневой URL (например, /api), который возвращает ссылки на все доступные коллекции верхнего уровня (/api/books, /api/authors). Это позволяет клиентам динамически обнаруживать ресурсы, не «зашивая» все пути в свой код.

### Сквозные задачи и Servlet Filters

Некоторые задачи, такие как логирование, безопасность, аудит или трассировка, не относятся к конкретной бизнес-логике, а применяются ко всему приложению. Их называют **сквозными задачами** (Cross-Cutting Concerns).

В Spring Boot для решения таких задач идеально подходят **Servlet Filters** (`jakarta.servlet.Filter`). Фильтр — это объект, который может перехватывать входящие HTTP-запросы до того, как они достигнут контроллера, и исходящие ответы после того, как контроллер их сформировал. Это позволяет нам добавить общую логику в одном месте, не изменяя десятки методов в контроллерах.

### Correlation ID и трассировка

**Correlation ID** (также известный как Request ID или Trace ID) — это уникальный идентификатор, который присваивается входящему запросу на границе системы. Этот ID затем передается с запросом через все микросервисы, которые участвуют в его обработке.

Когда вы смотрите в логи распределенной системы, вы видите перемешанные записи от сотен одновременных запросов. Correlation ID позволяет отфильтровать лог-записи, относящиеся только к одному конкретному запросу, и увидеть его путь через всю систему.

#### 1. Реализация:

- Создается фильтр, который проверяет наличие заголовка (например, X-Request-ID) в входящем запросе.
- Если заголовок есть, используется его значение.
- Если заголовка нет (запрос пришел извне), генерируется новый уникальный ID (обычно UUID).
- Этот ID добавляется в каждый ответ в виде заголовка, чтобы клиент тоже его знал.



## Mapped Diagnostic Context (MDC)

**MDC** — это инструмент библиотеки логирования (SLF4J/Logback), который позволяет обогащать все лог-сообщения, записанные в рамках одного потока, дополнительной контекстной информацией. Это, по сути, ThreadLocal Map<String, String>.

Вместо того чтобы вручную добавлять Correlation ID в каждое сообщение `log.info("ID: {}, Сообщение: ...", correlationId, "...")`, мы один раз помещаем его в MDC в начале обработки запроса. После этого фреймворк логирования автоматически добавит его во все последующие лог-сообщения в этом потоке.

В нашем фильтре после получения или генерации Correlation ID мы делаем `MDC.put("correlationId", id)`. **Критически важно** очищать MDC в блоке `finally`, чтобы избежать утечки контекста в другие запросы, которые могут быть обработаны этим же потоком позже.

### 3. Практический разбор

Мы доработаем наш сервис `demo-rest`, добавив точку входа HATEOAS, фильтр для логирования и фильтр для управления Correlation ID.

#### Шаг 1. Создание точки входа HATEOAS

В проекте `demo-rest` создадим новый контроллер, который будет отвечать за корневой эндпоинт `/api`.

**1. Создайте новый `RootController` в пакете `edu.rutmiit.demo.demorest.controllers`.**

**2. Добавьте следующий код. Он создает модель `RepresentationModel` и добавляет в нее ссылки на коллекции авторов и книг, используя `linkTo` и `methodOn` из Spring HATEOAS.**

```
@RestController
@RequestMapping("/api")
public class RootController {

    @GetMapping
    public RepresentationModel<?> getRoot() {
        RepresentationModel<?> rootModel = new
        RepresentationModel<>();
        rootModel.add(

linkTo(methodOn(AuthorController.class).getAllAuthors()).withRel(
    "authors"),

linkTo(methodOn(BookController.class).getAllBooks(null, 0,
    10)).withRel("books")
    );
    }
```



```
        return rootModel;
    }
}
```

### Шаг 2. Создание фильтра для логирования и трассировки

Мы создадим один универсальный фильтр, который будет выполнять обе задачи: управлять Correlation ID и логировать запрос.

1. Создайте новый пакет `edu.rutmiit.demo.demorest.filters`.

2. Создайте в нем класс `LoggingAndTracingFilter`.

3. Добавьте следующий код. Внимательно изучите комментарии, чтобы понять логику.

```
@Component
@Order(1) // Устанавливаем высокий приоритет, чтобы фильтр
сработал одним из первых
// Что значит 1 - чем меньше число, тем выше приоритет, фильтр
сработает раньше других фильтров,
// которые могут быть определены в приложении как в конвейере
public class LoggingAndTracingFilter implements Filter {

    private static final Logger log =
        LoggerFactory.getLogger(LoggingAndTracingFilter.class);
    private static final String CORRELATION_ID_HEADER = "X-
Request-ID";
    private static final String CORRELATION_ID_MDC_KEY =
        "correlationId";

    @Override
    public void doFilter(ServletRequest servletRequest,
        ServletResponse servletResponse, FilterChain filterChain)
        throws IOException, ServletException {

        HttpServletRequest request = (HttpServletRequest)
            servletRequest;
        HttpServletResponse response = (HttpServletResponse)
            servletResponse;

        // Управление Correlation ID - извлекаем из заголовка
        или генерируем новый
        String correlationId =
            request.getHeader(CORRELATION_ID_HEADER);
        if (!StringUtils.hasText(correlationId)) {
            correlationId = UUID.randomUUID().toString();
        }

        // Помещаем ID в MDC для обогащения всех логов
        MDC.put(CORRELATION_ID_MDC_KEY, correlationId);

        // Добавляем ID в заголовок ответа
```



## | GraphQL + HAL

```
response.setHeader(CORRELATION_ID_HEADER,
correlationId);

    long startTime = System.currentTimeMillis();

    try {
        // Логируем входящий запрос
        log.info("Request started: {} {}",
request.getMethod(), request.getRequestURI());

        // Передаем управление дальше по цепочке фильтров и
контроллеру
        filterChain.doFilter(request, response);

    } finally {
        long duration = System.currentTimeMillis() -
startTime;
        // Логируем завершение запроса
        log.info("Request finished: {} {} with status {} in
{}ms",
            request.getMethod(),
request.getRequestURI(), response.getStatus(), duration);

        // Критически важно! Очищаем MDC, чтобы избежать
утечки контекста
        MDC.remove(CORRELATION_ID_MDC_KEY);
    }
}
```

### Шаг 3. Настройка формата логов

Чтобы MDC заработал, нужно указать Logback, что мы хотим видеть correlationId в каждой строке лога.

1. Откройте файл application.properties в проекте demo-rest.
2. Добавьте следующую строку. Она определяет новый паттерн для вывода логов. %X{correlationId} — это специальный плейсхолдер для получения значения из MDC по ключу correlationId.

# Добавляем эту строку. Если такой ключ уже есть, заменяем его.

```
logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-
5level %logger{36} - [%X{correlationId}] - %msg%n
```

### Шаг 4. Запуск и тестирование

1. Перезапустите ваш сервис demo-rest.
2. Откройте терминал и выполните следующие команды, наблюдая за выводом в консоли IDE.

#### Тест 1. Точка входа HATEOAS



Выполните запрос к новому корневому эндпоинту <http://localhost:8080/api> и обратите внимание на новый ответ.

### Тест 2. Логирование и генерация Correlation ID

Выполните запрос к списку книг (<http://localhost:8080/api/books>) в Postman, либо в браузере, обратите внимание на заголовки.

В заголовках ответа вы увидите X-Request-ID с сгенерированным UUID.

В консоли IDE вы увидите примерно такие логи, где в квадратных скобках будет тот же ID.

### Тест 3. Передача существующего Correlation ID

Теперь симулируем запрос от другого микросервиса, который уже передал нам ID. В postman нужно добавить в запросе <http://localhost:8080/api/books> заголовок **X-Request-ID** с значением **my-custom-trace-id-42**

В заголовках ответа будет ваш ID, проверьте.

И в логах будет тот же ID, также проверяем.

#### 4. Задания для самостоятельной работы

##### Задание 1. Расширение точки входа

Модифицируйте RootController так, чтобы он также возвращал ссылку на Swagger UI документацию (/swagger-ui.html) с отношением (rel) "documentation".

##### Задание 2. Условное логирование

Модифицируйте LoggingAndTracingFilter. Добавьте условие, чтобы полные лог-сообщения Request started и Request finished выводились только для запросов, URI которых начинается с /api/. Для остальных запросов (например, к Swagger UI) логирование должно быть пропущено.

##### Задание 3. Фильтр для измерения производительности

Создайте новый фильтр PerformanceWarningFilter с @Order(2), чтобы он выполнялся после LoggingAndTracingFilter.

В этом фильтре замерьте время выполнения запроса (аналогично логгеру).

Если время выполнения превышает заданный порог (например, 20 миллисекунд), выведите в лог сообщение уровня WARN с информацией о медленном запросе. Например: WARN - Slow request detected: GET /api/books took 25ms.



### 5. Дополнительные материалы

#### **Ключевые аннотации и понятия:**

**@Component**

Регистрирует класс как компонент Spring, делая его доступным для сканирования и использования.

**@Order(n)**

Определяет порядок выполнения компонентов, если их несколько (фильтры, обработчики). Меньшее число означает более высокий приоритет.

**jakarta.servlet.Filter**

Интерфейс для создания фильтров в веб-приложениях Java.

**MDC (Mapped Diagnostic Context):** Механизм SLF4J для добавления контекстной информации в логи текущего потока.

**Важное замечание!** Всегда используйте блок try...finally при работе с MDC. Очистка контекста в блоке finally гарантирует, что correlationId от одного запроса случайно не "просочится" в логи другого запроса, если сервер приложений переиспользует тот же самый поток для обработки. Это критически важно для корректной работы в многопоточной среде.